

ES453 - Lista 1: Validador de CSV (CPF e cartão de crédito)

Casos de teste, decisões de implementação e justificativa do estilo do código

Aluno	João Antônio Dubeux
Disciplina	ES453 - Introdução a Python (UFPE) · Prof. Hermano Cabral
Data	03/10/2026
Arquivos	CSV.py (programa), casos_teste.csv (casos de teste), relatorio.html (relatório gerado) e este documento

1. Visão geral

O programa CSV.py lê um arquivo CSV sem cabeçalho, onde cada linha tem três campos (nome, CPF, número do cartão de crédito). Para cada linha ele verifica se os campos são não vazios, se o CPF tem 11 dígitos e passa no algoritmo de verificação do CPF, e se o cartão tem 16 dígitos e passa no algoritmo de Luhn. Os erros são impressos na tela, um por linha do relatório, no formato "linha NNN: CAMPO - motivo", com o número da linha em quatro posições. Como extra, o programa também grava relatorio.html com um resumo e a tabela de erros, para facilitar a conferência dos casos de teste.

Nenhuma biblioteca é importada, nem a standard library: o parser de CSV, os algoritmos de CPF e de Luhn e a geração do relatório foram escritos com as construções da própria linguagem.

Como executar: `python CSV.py` e, quando pedido, digitar o nome do arquivo (por exemplo `casos_teste.csv`).

Resultado geral dos testes: 28 linhas lidas (27 com dados e 1 em branco). Há 9 linhas válidas, 18 linhas com erro e 19 erros no total, porque a linha 21 tem erro no CPF e no cartão ao mesmo tempo. Por tipo: 4 campos vazios, 5 caracteres inválidos, 5 quantidades incorretas de dígitos e 5 erros nos dígitos verificadores.

2. Por que list comprehensions, geradores, enumerate, zip e afins em vez do estilo C

Em C, quase tudo que o programa faz exigiria um laço for com índice, vetores de tamanho fixo, variáveis acumuladoras e vários if. Em Python, as construções usadas descrevem o que se quer obter e não como controlar o índice. Isso é preferível pelos motivos abaixo, com exemplos tirados do próprio código.

- **List comprehension.** Em `nums = [int(c) for c in digits]` e `digits = [c for c in value if c in DIGITS]`, a conversão e o filtro cabem em uma linha. Em C seria preciso declarar um vetor com tamanho máximo, fazer um laço com índice, converter com `c - '0'` e controlar manualmente quantos elementos foram preenchidos. Um campo maior que o vetor causaria estouro de buffer; em Python a lista cresce sozinha.
- **zip, range com passo negativo e expressão geradora.** O dígito verificador do CPF é `sum(j * k for j, k in zip(base, range(len(base) + 1, 1, -1)))`. Os pesos (10 a 2, depois 11 a 2) são gerados e emparelhados com os dígitos sem nenhum índice. O equivalente em C tem um índice, uma variável de peso que decresce e um acumulador inicializado à mão, e é exatamente aí que ocorrem os erros de off-by-one. A soma consome a expressão geradora um termo por vez, sem criar uma lista intermediária.

- **enumerate e reversed.** O Luhn precisa percorrer os dígitos da direita para a esquerda e saber se a posição é par ou ímpar. `enumerate(reversed(digits))` entrega posição e dígito juntos, dispensando o contador `i++` e o cálculo `n - 1 - i` de um laço em C. Em `find_errors`, `enumerate(lines, start=1)` numera as linhas a partir de 1 sem contador manual.
- **any().** `any(c not in DIGITS and c not in separators for c in value)` para no primeiro caractere inválido. Em C seria preciso uma variável de sinalização e um `break`.
- **Geradores (yield e yield from).** `find_errors` e `report` entregam um erro de cada vez, sob demanda. Em C seria necessário alocar dinamicamente um vetor de estruturas de tamanho desconhecido (com `realloc`) ou misturar a lógica de validação com a impressão. Aqui o mesmo gerador alimenta a saída na tela e o relatório HTML, separando validação de apresentação. Como o gerador só pode ser percorrido uma vez, o programa guarda as linhas do arquivo em uma lista e gera os erros de novo quando precisa.
- **Fatiamento e preenchimento.** `split(line) + ["", "", ""]` seguido de `[:3]` garante sempre três campos. Uma linha com campos faltando vira campos vazios, e campos a mais são descartados. Em C, ler o terceiro campo de uma linha que só tem dois seria acesso fora do vetor.
- **Strings.** `"".join(...)` e f-strings substituem `strcat` e `sprintf`, que exigem buffers com tamanho calculado. O caso de teste de 300 dígitos (linha 27) é o tipo de entrada que estouraria um buffer fixo em C.
- **Funções como argumento.** `validate` recebe a função de verificação (`checker`), então a mesma função valida o CPF e o cartão. Em C seria um ponteiro para função, com sintaxe mais pesada.
- **with open.** O arquivo é fechado automaticamente, mesmo se ocorrer erro, sem precisar lembrar de um `fclose` em cada caminho de saída.

Em resumo, o código fica menor, tem menos estado mutável e menos pontos onde errar um índice ou um tamanho de vetor, e cada função pode ser testada separadamente. O custo é um desempenho um pouco menor, que não importa para este problema.

3. Decisões de implementação

- 1 **Linhas em branco.** São ignoradas, mas a numeração continua correta, pois a contagem é feita pelo `enumerate` antes do filtro (caso da linha 7).
- 2 **Parser de CSV próprio.** Como o módulo `csv` é proibido, `split` percorre a linha caractere a caractere: as aspas ligam e desligam um modo em que a vírgula não separa campos, o que permite nomes como "Silva, Maria". Limitação conhecida: o arquivo é lido linha a linha, então um campo entre aspas que contenha uma quebra de linha é tratado como duas linhas.
- 3 **Sempre três campos.** Campos faltando viram vazios e campos extras são descartados (função `extract`).
- 4 **Somente dígitos ASCII.** O programa compara com a string "0123456789" em vez de usar `isdigit()`, porque `isdigit()` e `int()` aceitam dígitos de outros alfabetos, como os de largura total, que passariam despercebidos.
- 5 **Separadores.** No CPF são aceitos ".", "-", e "/", como pede a lista. No cartão é aceito apenas o espaço. Um cartão com hífen é considerado inválido.
- 6 **Espaços nas pontas.** Espaços antes e depois de cada campo são removidos (`strip`).
- 7 **Ordem das verificações em cada campo.** Vazio, depois caractere inválido, depois quantidade de dígitos e por último os dígitos verificadores. Cada campo gera no máximo um erro, de modo que uma linha com problema no CPF e no cartão gera dois erros no relatório.

- 8 **Tipos de erro além dos três da lista.** A lista prevê vazio, quantidade incorreta e dígitos verificadores. O programa acrescenta dois: "caractere inválido", para separar "o campo tem letras ou símbolos" de "o campo tem poucos dígitos" (é mais útil para quem corrige o arquivo e distingue o caso de uma pessoa que digitou o nome no lugar do CPF), e "NOME - campo vazio", porque a lista exige que todos os campos sejam não vazios mas não define mensagem para o nome. Na leitura estrita da lista, um CPF como 123.456.789-AB seria reportado como quantidade incorreta de dígitos (9 dígitos).
- 9 **CPF com todos os dígitos iguais.** Valores como 111.111.111-11 passam na conta dos dígitos verificadores, mas não são CPFs válidos; por isso são reportados como erro nos dígitos verificadores.
- 10 **Cartão 0000 0000 0000 0000.** A lista define cartão válido como 16 dígitos mais Luhn, e esse número satisfaz as duas condições. Ele é aceito, e fica registrado como limitação: o programa não consulta a bandeira nem a existência do cartão.
- 11 **Contagem de dígitos.** O NN da mensagem de quantidade incorreta conta apenas os dígitos do campo, sem os separadores.
- 12 **Relatório HTML.** É um extra. O texto de cada linha é escapado (esc) para que conteúdo como <script> vindo do CSV não seja interpretado como HTML, e é cortado em 60 caracteres (short) para não estourar a tabela. O relatório da tela mostra tudo.

4. Casos de teste: entrada, resultado esperado e resultado obtido

Todos os casos estão em casos_teste.csv, e "Linha" é o número da linha nesse arquivo. "Esperado" segue a redação da lista (campo, hífen e motivo). "Obtido" é a saída literal do programa. A categoria do erro e o número da linha coincidem em todos os casos; o texto literal do programa usa minúsculas, escreve "caracteres" no lugar de "dígitos" na mensagem de quantidade e diz "erro de verificação" no lugar de "erro nos dígitos verificadores".

Linha	Entrada	Esperado (lista)	Obtido (programa)
1	Maria da Silva,529.982.247-25,4539 7911 3571 6476	Sem erro	Nenhuma linha no relatório
2	José Oliveira,39053344705,5522000 011112222	Sem erro	Nenhuma linha no relatório
3	Ana Souza,111.444.777-35,41111111 11111111	Sem erro	Nenhuma linha no relatório
4	Carlos Lima,,4539 7911 3571 6476	Linha 4: CPF - Campo vazio	linha 4: CPF - campo vazio
5	Paula Mendes,529.982.247-25,	Linha 5: CARTÃO - Campo vazio	linha 5: cartao - campo vazio
6	,390.533.447-05,5522 0000 1111 2222	Linha 6: NOME - Campo vazio	linha 6: NOME - campo vazio
7	(linha em branco)	Ignorada; a numeração continua	Nenhuma linha no relatório; a linha seguinte é reportada como 8
8	Dez Dígitos,529.982.247-2,4539 7911 3571 6476	Linha 8: CPF - Quantidade incorreta de dígitos (10 dígitos)	linha 8: CPF - quantidade incorreta de caracteres (10 dígitos)
9	Doze Dígitos,529.982.247-255,4539 7911 3571 6476	Linha 9: CPF - Quantidade incorreta de dígitos (12 dígitos)	linha 9: CPF - quantidade incorreta de caracteres (12 dígitos)
10	CPF Dígito Errado,529.982.247-26,4539 7911 3571 6476	Linha 10: CPF - Erro nos dígitos verificadores	linha 10: CPF - erro de verificação
11	CPF Repetido,111.111.111-11,4539 7911 3571 6476	Linha 11: CPF - Erro nos dígitos verificadores	linha 11: CPF - erro de verificação
12	Dona Maria Aparecida,Maria Aparecida,4539 7911 3571 6476	Linha 12: CPF - Caractere inválido (decisão 8)	linha 12: CPF - caractere invalido
13	CPF Letras,123.456.789-AB,4539 7911 3571 6476	Linha 13: CPF - Caractere inválido (decisão 8)	linha 13: CPF - caractere invalido

Linha	Entrada	Esperado (lista)	Obtido (programa)
14	Separadores Misturados,529-982/247.25,4539 7911 3571 6476	Sem erro	Nenhuma linha no relatório
15	Cartao 15 Digitos,529.982.247-25,4539 7911 3571 647	Linha 15: CARTÃO - Quantidade incorreta de dígitos (15 dígitos)	linha 15: cartao - quantidade incorreta de caracteres (15 dígitos)
16	Cartao 17 Digitos,529.982.247-25, 45397911357164768	Linha 16: CARTÃO - Quantidade incorreta de dígitos (17 dígitos)	linha 16: cartao - quantidade incorreta de caracteres (17 dígitos)
17	Cartao Luhn Errado,390.533.447-05,5522 0000 1111 2223	Linha 17: CARTÃO - Erro nos dígitos verificadores	linha 17: cartao - erro de verificação
18	Cartao com Hifen,529.982.247-25,4 539-7911-3571-6476	Linha 18: CARTÃO - Caractere inválido (decisão 8)	linha 18: cartao - caractere invalido
19	Cartao Texto,529.982.247-25,nao tenho cartao	Linha 19: CARTÃO - Caractere inválido (decisão 8)	linha 19: cartao - caractere invalido
20	Cartao Zeros,529.982.247-25,0000 0000 0000 0000	Sem erro (decisão 10)	Nenhuma linha no relatório
21	Tudo Errado,111.444.777-36,4111 1111 1111 1112	Duas linhas: CPF - Erro nos dígitos verificadores e CARTÃO - Erro nos dígitos verificadores	linha 21: CPF - erro de verificação e linha 21: cartao - erro de verificação
22	Largura Total,529.982.247-25,[16 dígitos de largura total, U+FF10 a U+FF19, em grupos de 4]	Linha 22: CARTÃO - Caractere inválido (decisão 4)	linha 22: cartao - caractere invalido
23	"Silva, Maria",390.533.447-05,5522 0000 1111 2222	Sem erro	Nenhuma linha no relatório
24	"=HYPERLINK("http://x.com")",52 9.982.247-25,4539 7911 3571 6476	Sem erro (o nome é só texto)	Nenhuma linha no relatório
25	'; DROP TABLE clientes;-- ,529.982.247-25,4539 7911 3571 6476	Sem erro (o nome é só texto)	Nenhuma linha no relatório
26	Campo Faltando,529.982.247-25	Linha 26: CARTÃO - Campo vazio	linha 26: cartao - campo vazio
27	Longo, seguido de 300 dígitos 1 e ,4539 7911 3571 6476	Linha 27: CPF - Quantidade incorreta de dígitos (300 dígitos)	linha 27: CPF - quantidade incorreta de caracteres (300 dígitos)
28	Espacos Nas Pontas , 529.982. 247-25 , 4539 7911 3571 6476	Sem erro	Nenhuma linha no relatório

Os CPFs e os cartões válidos dos casos de teste foram gerados aplicando os próprios algoritmos, de modo que os casos "sem erro" são realmente válidos e os casos de erro mudam apenas o que se quer testar.

5. Por que cada caso foi escolhido e que erro de implementação ele procura detectar

- **Linhas 1 a 3 (controles válidos).** Mostram que o programa não gera falso positivo. Usam CPFs e cartões diferentes, com e sem pontuação e com e sem espaços, para detectar um validador que só aceita um formato ou que rejeita tudo.
- **Linha 4 (CPF vazio).** Simula uma pessoa que pulou o campo. Detecta quebra com `int("")` ou a mensagem errada ("quantidade incorreta" em vez de "campo vazio").
- **Linha 5 (cartão vazio no fim da linha).** A linha termina em vírgula. Detecta parsers que descartam o último campo vazio ou geram `IndexError`.
- **Linha 6 (nome vazio).** A lista exige que todos os campos sejam não vazios. Detecta o esquecimento de validar o nome e erro de posição dos campos.
- **Linha 7 (linha em branco).** Detecta quebra com linha vazia e, principalmente, numeração desalinhada se as linhas em branco fossem removidas antes de numerar.

- **Linhas 8 e 9 (CPF com 10 e 12 dígitos).** Testam os dois lados do limite. Detectam comparações com $\geq$ ou $\leq$ em vez de $=$ e erros de off-by-one. O 12º dígito simula alguém que digitou um dígito a mais.
- **Linha 10 (último dígito do CPF alterado).** O CPF tem 11 dígitos, então só falha no dígito verificador. Detecta implementações que conferem apenas o tamanho ou só o primeiro dígito verificador.
- **Linha 11 (CPF com dígitos repetidos).** Passa na conta, mas não é um CPF válido. Detecta a falta da regra dos dígitos todos iguais.
- **Linha 12 (nome no lugar do CPF).** É o exemplo da lista: uma pessoa com pouca experiência digita o nome no campo errado. Detecta `ValueError` do `int()` e mensagem enganosa.
- **Linha 13 (CPF com letras no meio).** Mistura dígitos e letras, o que confunde a contagem (só 9 dígitos). Detecta implementações que tentam converter tudo em número ou que contam letras como dígitos.
- **Linha 14 (separadores misturados).** A lista diz que os separadores ".", "-", e "/" são opcionais. Detecta implementações que exigem a máscara padrão 000.000.000-00.
- **Linhas 15 e 16 (cartão com 15 e 17 dígitos).** Também testam os dois lados do limite de 16. Simulam a omissão de um dígito (15) e o dígito a mais (17), erros típicos de digitação. Detectam off-by-one e confusão entre contar caracteres e contar dígitos.
- **Linha 17 (cartão com Luhn errado).** Tem 16 dígitos, então só falha no Luhn. Detecta a falta do algoritmo ou o dobro aplicado nas posições erradas (sentido do percurso).
- **Linha 18 (cartão com hífen).** Quem copia o número de um extrato costuma usar hífen, e a lista só permite espaços. Detecta código permissivo que remove qualquer caractere que não seja dígito.
- **Linha 19 (texto no cartão).** Simula uma pessoa que escreve "não tenho cartão". Detecta quebra no `int()` e falha ao reportar o campo certo.
- **Linha 20 (cartão só de zeros).** Passa no Luhn e tem 16 dígitos, então deve ser aceito segundo a lista. Detecta rejeição arbitrária e documenta a limitação (decisão 10).
- **Linha 21 (CPF e cartão errados na mesma linha).** A lista exige dois erros no relatório. Detecta um `return` ou `break` depois do primeiro erro.
- **Linha 22 (dígitos de largura total).** Entrada que pode vir de um teclado configurado em outro modo ou de um atacante. `isdigit()` e `int()` aceitam esses caracteres; o programa deve rejeitá-los. Detecta o uso de `isdigit()`.
- **Linha 23 (nome com vírgula entre aspas).** Detecta o uso de `split(",")`, que produziria quatro campos e deslocaria CPF e cartão.
- **Linha 24 (nome que começa com "=" e parece fórmula, "CSV injection").** Um atacante tenta fazer um programa de planilha ou o próprio programa interpretar o dado como comando. Detecta o uso de `eval` e confirma que o nome é tratado como texto. O dado também aparece escapado no relatório HTML.
- **Linha 25 (texto de SQL injection com aspa simples).** Testa se o programa trata o conteúdo apenas como texto e se o apóstrofo e o ponto e vírgula não atrapalham o parser, já que só a aspa dupla é delimitadora. Detecta montagem de comandos a partir de dados de entrada.
- **Linha 26 (linha com apenas dois campos).** Simula um arquivo mal exportado. Detecta `IndexError` ao acessar o terceiro campo.
- **Linha 27 (campo com 300 dígitos).** Entrada gigante, típica de quem tenta quebrar o programa. Em C estouraria um buffer fixo. Detecta limites de tamanho, lentidão e quebra do relatório (o HTML corta o texto em 60 caracteres).
- **Linha 28 (espaços nas pontas dos campos).** Pessoas costumam deixar espaços antes ou depois do valor. Detecta a falta do `strip`, que faria um CPF válido ser rejeitado por causa de um espaço.

6. Conclusão

Os 28 casos do arquivo `casos_teste.csv` (27 linhas de dados e 1 linha em branco) se comportaram como esperado: as 9 linhas válidas não geraram erro e as 18 linhas inválidas geraram os 19 erros previstos, com o número de linha e a categoria corretos. Como limitações conhecidas, o programa não trata campos entre aspas que contenham quebra de linha e não verifica se o CPF ou o cartão existem de fato, apenas se são consistentes com os algoritmos de verificação.